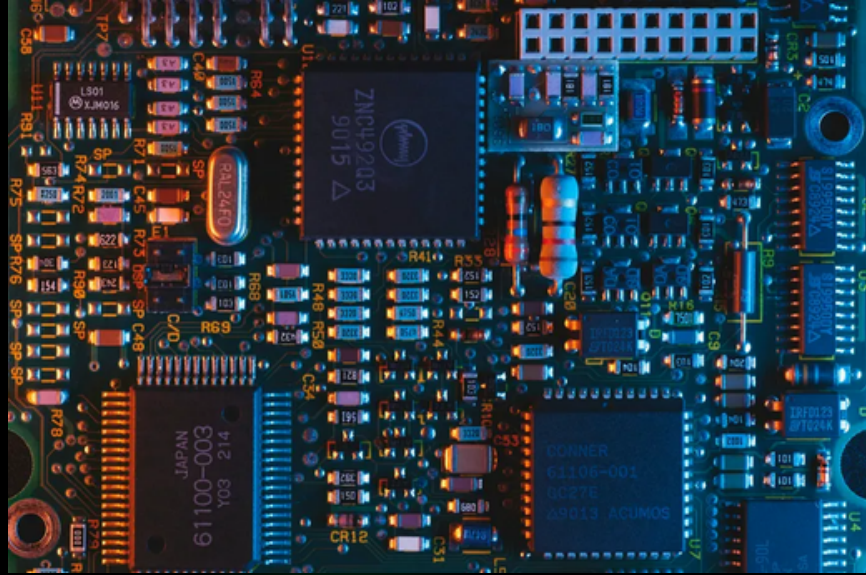


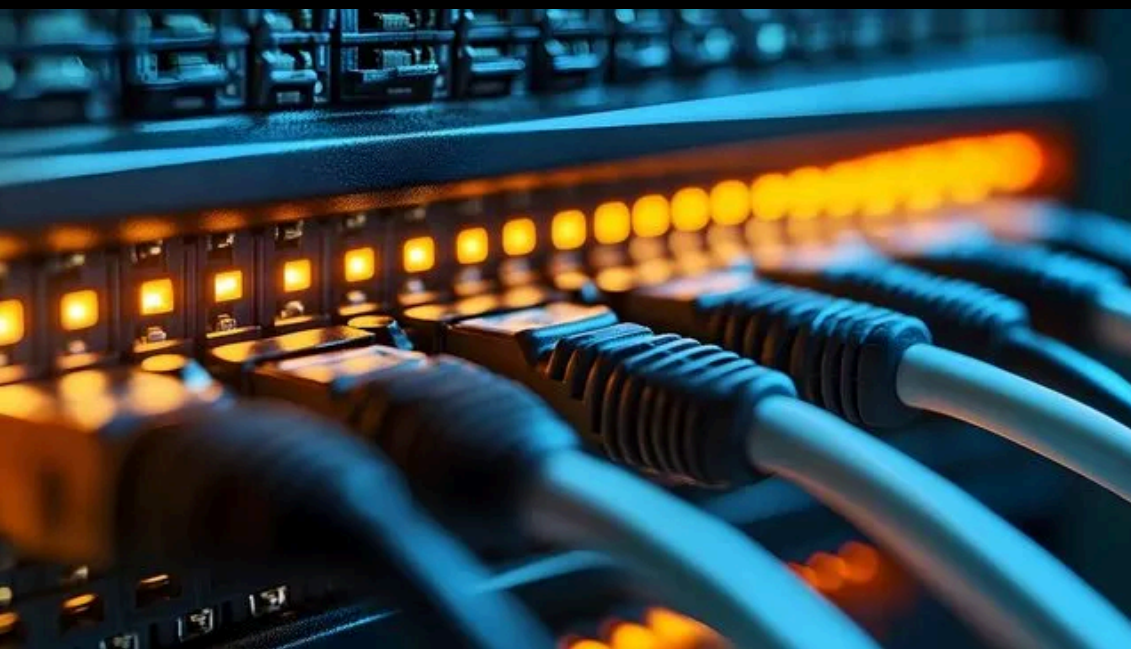
V 1.0.0



ECHIDNA PROJECT

NORMES DOCUMENT

BY : YASSIR ZAYD CHEHABI



WHAT ECHIDNA-NORMS ACTUALLY COVERS

- 1 ■ PURPOSE & SCOPE
- 2 ■ NAMING CONVENTIONS
- 3 ■ PROJECT STRUCTURE && DOCUMENTATION STANDARD
- 4 ■ GIT & COMMIT CONVENTIONS
- 5 ■ STATUS/RESPONSE CODE SYSTEM
- 6 ■ VERSIONING RULES
- 7 ■ DEFINITION OF DONE
- 8 ■ SECURITY/TRUST BASELINE

Notice that this document serves as a general **norm** and **philosophy baseline**. It can be adapted for specific projects and may be edited accordingly. Your reviews and contributions will help us make everything more **functional** and **effective**.

PURPOSE & SCOPE

1.1 — START

 Echidna-Norms is the root standard every project under the Holy Graph inherits from. It exists so that no matter which circle you're looking at — language, CPU, networking, OS — it reads as one coherent system built by one mind, not a pile of disconnected experiments.

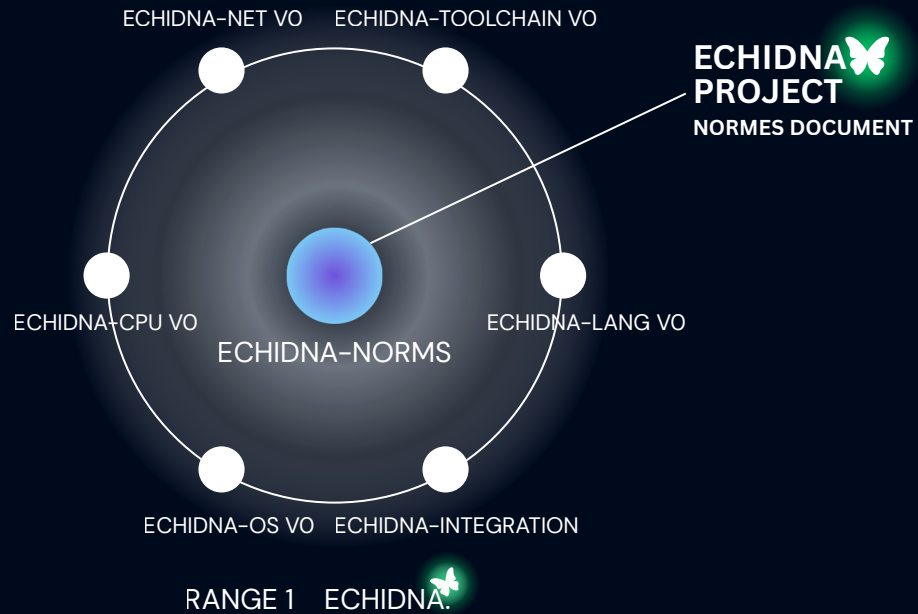
This document defines what's universal: naming, structure, commit conventions, documentation format, the status/response code system, versioning rules, and the Definition of Done every circle must pass before it's called finished. It does not define project-specific rules — Echidna-Lang's grammar, Echidna-Net's protocol semantics, Echidna-CPU's instruction set each get their own norms file later, built on top of this one, not instead of it.

The philosophy behind why this exists at all: most of the tech we live on today runs on protocols and conventions designed decades ago, for constraints and threats that don't exist anymore — patched forward instead of rethought. We're not interested in inheriting that ceiling. Echidna starts from the assumption that the systems underneath our lives deserve to be built with the same ambition as the interfaces on top of them — real cyberpunk-grade engineering, not 1980s logic wearing a modern coat of paint. Every norm in this document exists to make that possible: consistent enough to build fast, disciplined enough to actually finish, and deliberate enough that nothing here is inherited by accident. 

ECHIDNA PROJECT

1.1 — START

THE HOLIEST GRAPH EVER



V:1
MAXIMUM: RANGE 1

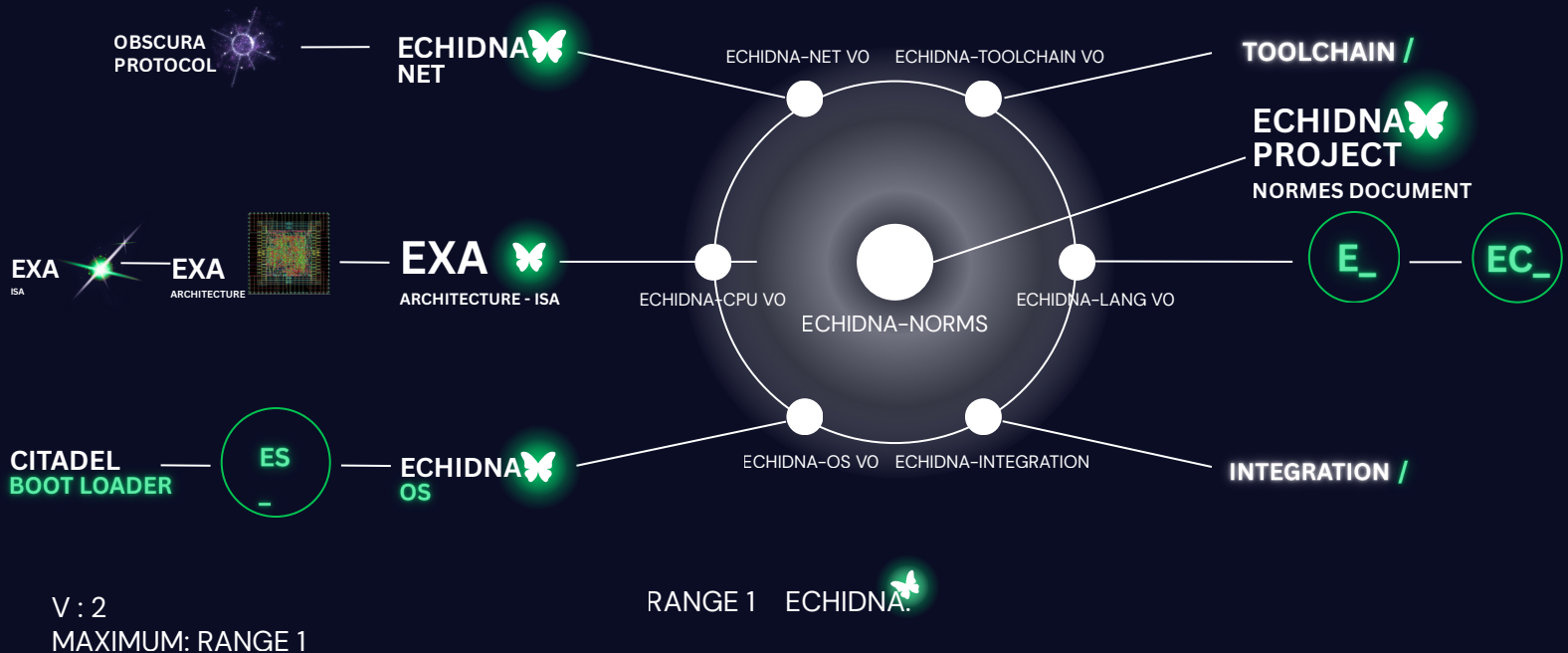
YOU ARE AT “ECHIDNA-NORMS”

This graph describes the project structure — starting from the center and expanding outward to the rims, until the circle is fully complete, project by project.

ECHIDNA PROJECT

1.1 — START

THE HOLIEST GRAPH EVER V2

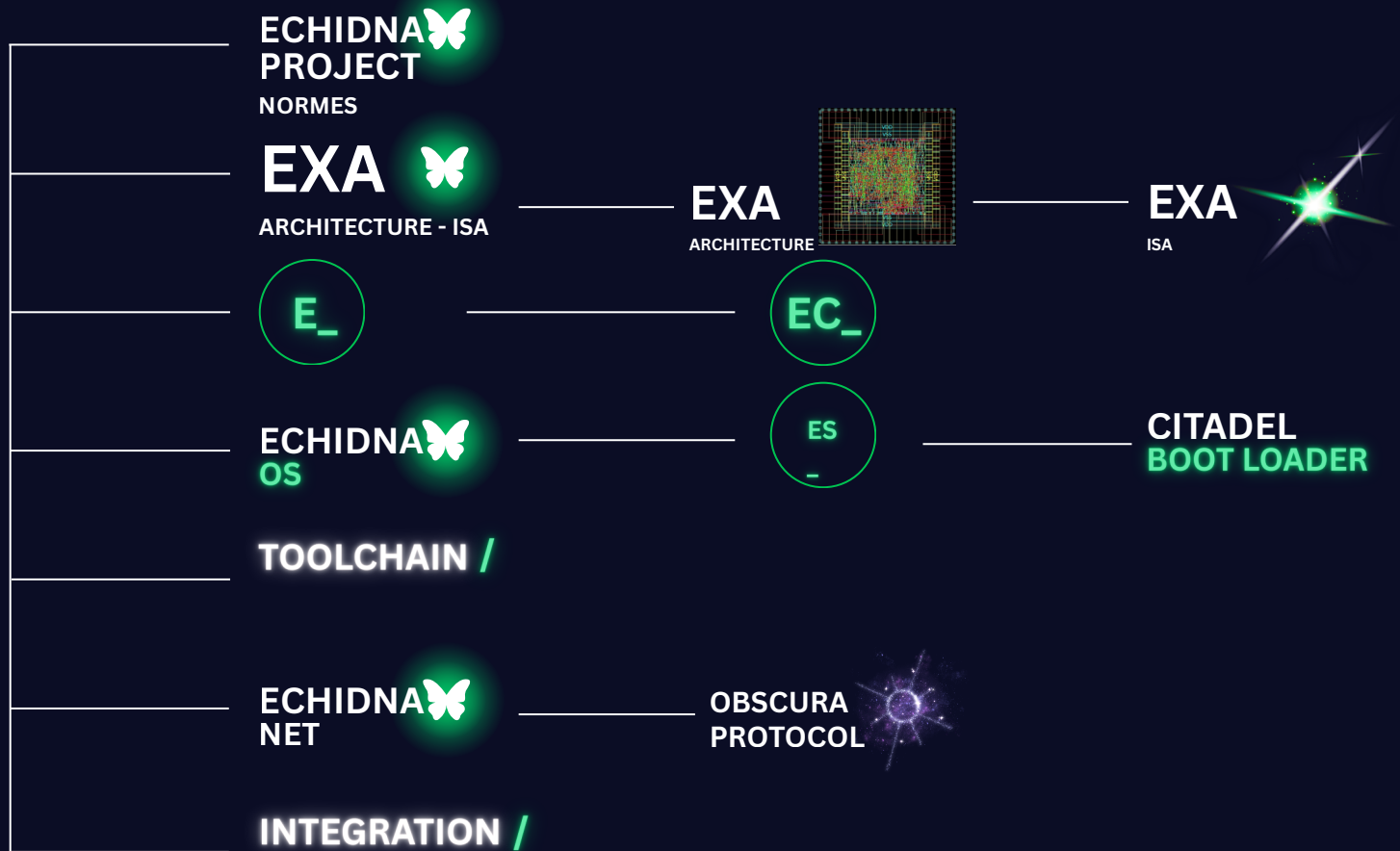


This graph describes the project structure — starting from the center and expanding outward to the rims, until the circle is fully complete, project by project.

ECHIDNA PROJECT

1.1 — START

THE HOLY TREE



This graph describes the project structure — starting the top to bottom, until the tree is fully complete, project by project.

ECHIDNA PROJECT

1.1 ■ — START

```
ECHIDNA/  
├── NORMS/      # SPECIFICATIONS & ARCHITECTURE DOCS  
├── CPU/        # CUSTOM CPU / EMULATOR / ISA IMPLEMENTATION  
├── LANG/       # COMPILER / LEXER / PARSER  
├── OS/         # KERNEL SOURCE & BOOTLOADER  
├── TOOLCHAIN/  # ASSEMBLER, LINKER, TOOLS  
├── NET/        # NETWORKING PROTOCOL STACK  
├── INTEGRATION/ # SYSTEM TESTS & FULL-STACK INTEGRATION  
├── .GITIGNORE  # IGNORES BUILD ARTIFACTS (.O, BINARIES, BUILD/)  
├── LICENSE     # OPEN-SOURCE LICENSE (MIT/GPL)  
└── README.MD   # THE MAIN LANDING PAGE
```

This graph describes the projects — starting the top to bottom, until the tree is fully complete, project by project.

NAMING CONVENTIONS

2.1 ■ — VARIABLES

■ ALL INDEXES MUST FOLLOW THIS PATTERN:

```
INT IN; // if you gonna use 1 indexes in fonction
```

```
INT I, J, W, X, Z, Z; // if you gonna use 5 indexes max in each fonction start from i to z
```

```
INT IN_1, IN_99; // if you gonna use many indexes in fonction do: DATA_IN_NUMBER;
```

■ ALL DATATYPE INCLUDING STRINGS AND ARRAYS VARIABLES FOLLOWS THIS PATTERS:

```
INT LABEL_LABEL; // examples are: INT FIRST_USER, SECOND_TABLE, FIRST_INPUT;
```

2.2 ■ — FONCTIONS

■ ALL FONCTIONS NAME FOLLOWS THIS PATTERS:

```
DATA_TYPE E_NAME( INPUTS) // examples are: VOID E_FILE_NAME_PRINTER(VOID);
```

▲ ALL VARIABLE ARE UPERCASE LETTERS SAME GOES FOR FONCTIONS.

2.3 ■ — ALL VARIABLE MUST BE CAPITAL FOR EASIER DEBUG

NOTHING MUCH YOU HAVE FREEDOME OVER YOUR CODING STYLE



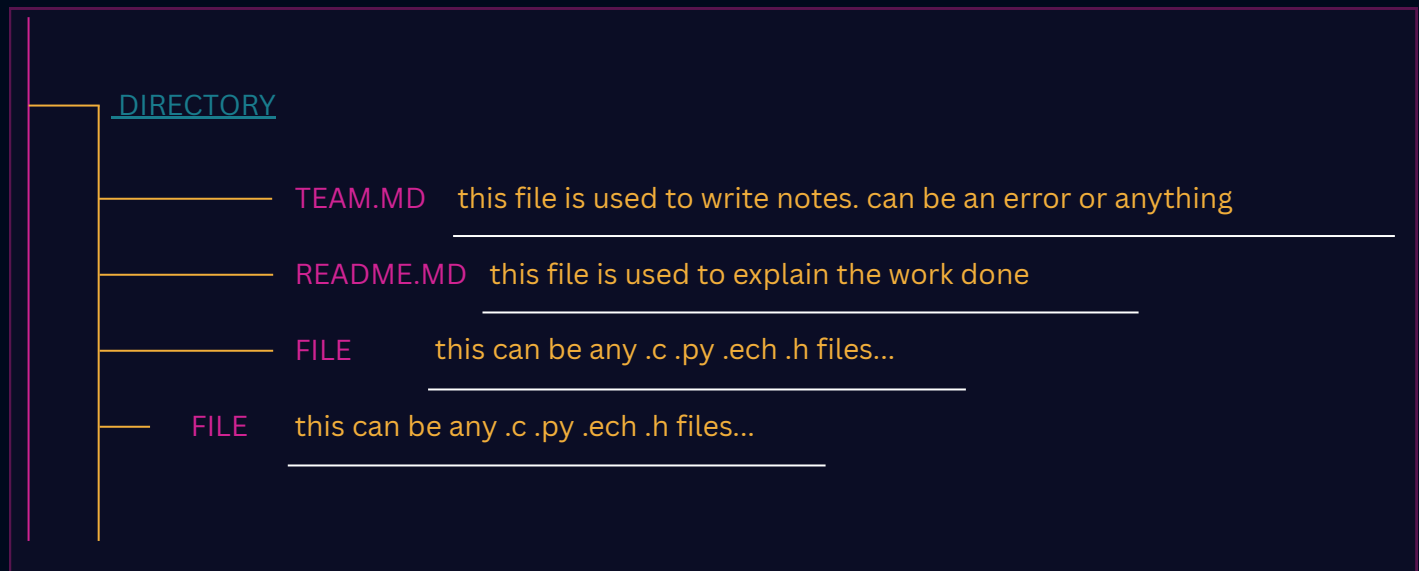
PROJECT STRUCTURE & DOCUMENTATION STANDARD



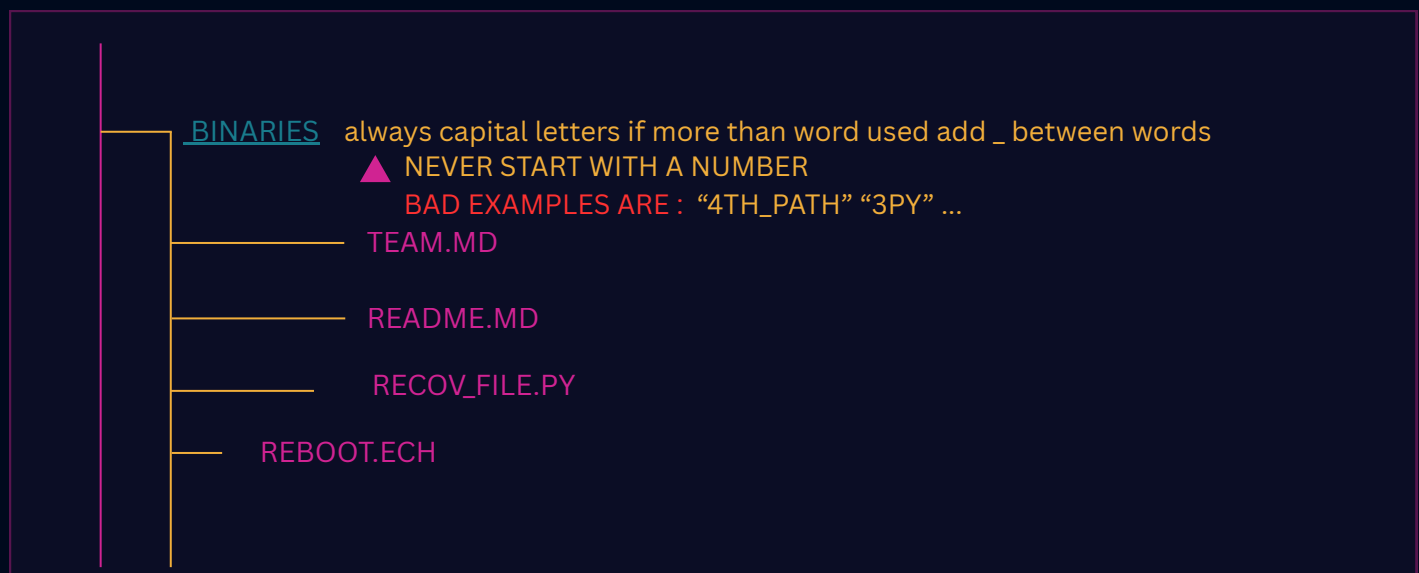
3.1 — FILES STRUCTURE

ALL PROJECTS FOLLOWS THIS TREE (EXCEPTION ARE MENTIONED IN THE PROJECT READ.ME OR COMMENTS):

ALL PROJECTS FOLLOWS THIS TREE (EXCEPTION ARE MENTIONED IN THE PROJECT READ.ME OR COMMENTS):



ALL FILES && DIRECTORIES MUST HAVE NAME AS PROTOTYPED:



PROJECT STRUCTURE & DOCUMENTATION STANDARD



3.1 — FILES STRUCTURE

THE ECHIDNA REPO STRUCTURE:

```
ECHIDNA/  
├── NORMS/      # SPECIFICATIONS & ARCHITECTURE DOCS  
├── CPU/        # CUSTOM CPU / EMULATOR / ISA IMPLEMENTATION  
├── LANG/       # COMPILER / LEXER / PARSER  
├── OS/         # KERNEL SOURCE & BOOTLOADER  
├── TOOLCHAIN/  # ASSEMBLER, LINKER, TOOLS  
├── NET/        # NETWORKING PROTOCOL STACK  
├── INTEGRATION/ # SYSTEM TESTS & FULL-STACK INTEGRATION  
├── .GITIGNORE  # IGNORES BUILD ARTIFACTS (.O, BINARIES, BUILD/)  
├── LICENSE     # OPEN-SOURCE LICENSE (MIT/GPL)  
└── README.MD  # THE MAIN LANDING PAGE
```

PROJECT STRUCTURE & DOCUMENTATION STANDARD

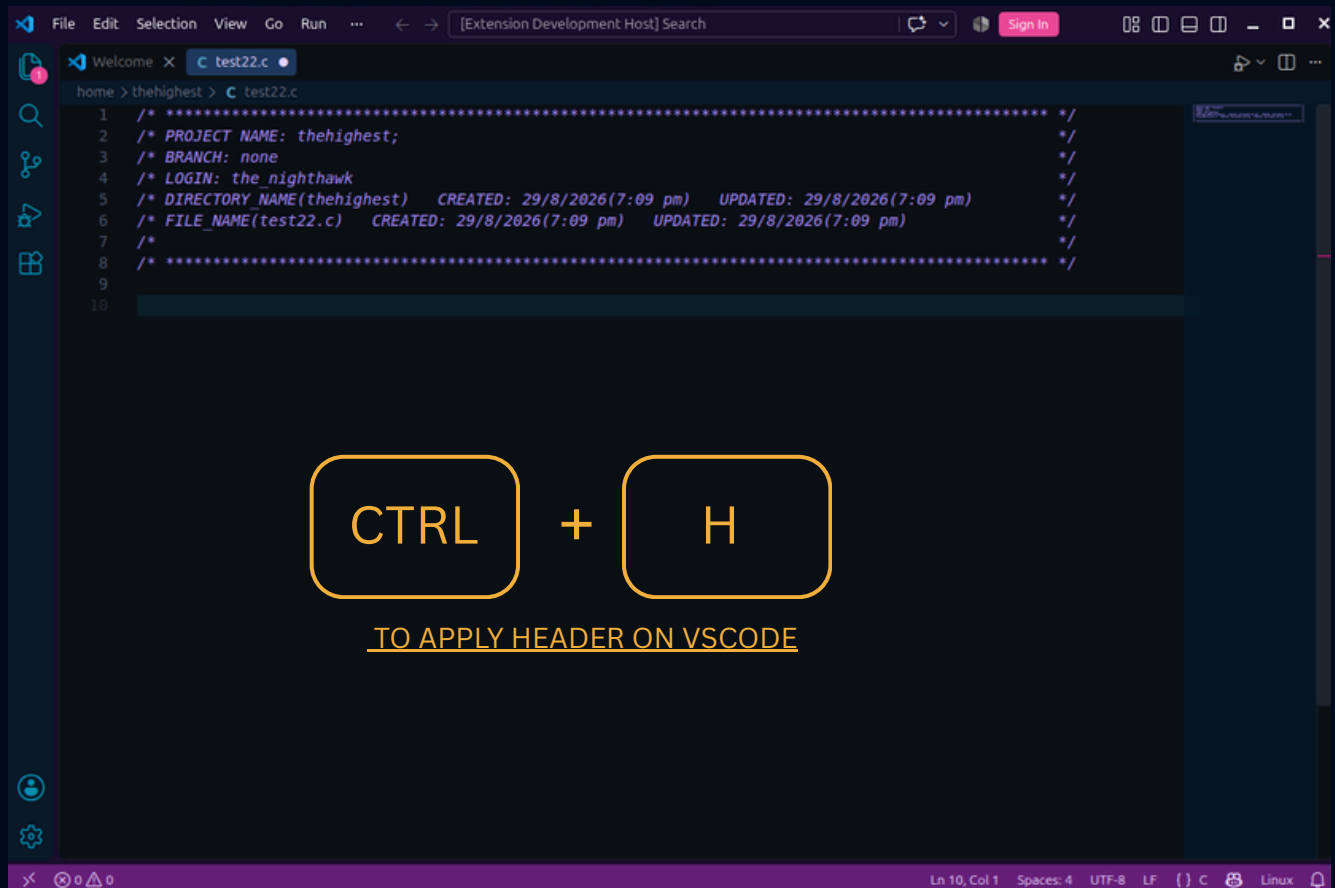


3.2 — DOCUMENTATION STANDARD

- ▲ EVERY FILE MUST HAVE THE FOLLOWING HEADER (SPECIALLY FOR CODE FILES)
- ▲ THIS HEADER WILL BE APPLIED AUTOMATICALLY ON YOUR VSCODE OR VIM PROJECT IS ON GITHUB

```
/* ***** */
/*      PROJECT NAME: E_PROTOCOL;                               */
/*      BRANCHE:                                                */
/*      LOGIN: THE_NIGHTHAWK                                   */
/*      DIRECTORY_NAME(XXXXXXX)  CREATED: 24/5/2099(10:00 AM)  UPDATED 24/5/2099 (10:30 PM) */
/*      FILE_NAME(XXXXXXX)      CREATED: 24/5/2099(10:02 AM)  UPDATED 24/5/2099 (10:30 PM) */
/*      ***** */
```

- ▲ THIS IS WHAT IT LOOKS LIKE ON YOUR VS CODE. HEADER APPLIED BY “CTRL + H”



PROJECT STRUCTURE & DOCUMENTATION STANDARD



■ HOW TO SET UP THE HEADER FOR VSCODE USERS ?

▲ YOU CAN PICK ONE OF THE FOLLOWING WAYS:

METHOD 1

FOR UBUNTU USERS VSCODE



- 1 ■ download the official "echidna_header_vscode_.zip" file from our repo
- 2 ■ extract the .zip file
- 3 ■ apply the following comans inside the "ECHIDNA_HEADER" foolder

```
npm install -g @vscode/vsce
```

```
vsce package
```

```
code --install-extension echidna-header-0.1.0.vsix
```

- 4 ■ reboot vscode if used. now the extention is working locally!



<success>

PROJECT STRUCTURE & DOCUMENTATION STANDARD



METHOD 2

FOR UBUNTU USERS VSCODE



- 1 ■ download the official extension on vscode and here you are 

<success>

■ HOW TO SET UP THE HEADER FOR VIM / NVIM USERS ?

METHOD 1

FOR VIM NVIM USERS



- 1 ■ download the official "echidna_header.vim" file from our repo
- 2 ■ Drop it in ~/.vim/plugin/  or ~/.config/nvim/plugin/  and restart.
- 3 ■ **Ctrl+H** in normal mode inserts the header at the top of the file, pulling in the git branch, directory name, filename, and timestamps automatically.
- 4 ■ On every **:w**, it auto-refreshes just the **UPDATED** timestamp on the FILE_NAME line — same behavior as the **VSCode** extension.
- 5 ■ **g:echidna_login** and **g:echidna_project_name** can be overridden in your .vimrc/init.vim if you want a fixed project name instead of the **E_PROTOCOL** default.

<success>

One thing to flag: this fills in BRANCHE from git rev-parse — if the file isn't in a git repo yet it'll just leave that field blank, which matches how your norms doc shows it



PROJECT STRUCTURE & DOCUMENTATION STANDARD



▲ EVERY FILE MUST (CODE) CANNOT BE ACCEPTABLE IF IT REPRODUCE ANY WARNINGS

■ HOW EVERY TEAM.MD MUST LOOK LIKE:

```
/* ***** */
/*      PROJECT NAME: E_PROTOCOL;                      */
/*      BRANCHE:                                         */
/*      LOGIN: THE_NIGHTHAWK                           */
/*      DIRECTORY_NAME(XXXXXXX)  CREATED: 24/5/2099(10:00 AM)  UPDATED 24/5/2099 (10:30 PM) */
/*      FILE_NAME(Team.MD)      CREATED: 24/5/2099(10:02 AM)  UPDATED 24/5/2099 (10:30 PM) */
/* ***** */
```

<NOTES TO THE TEAM>

TREE NAME:
DIRECTORY NAME:
FILE NAME:

__GENRAL NOTES__

__BUGS AND ERROS TO REPORT__

BUG (1)
LINE: XX (COMMENTS HERE ALSO MUST BE COMMENTS IN THE BUG(S) LINE(S))

BUG (2)
FROM LINE: XX TO LINE XX

PROJECT STRUCTURE && DOCUMENTATION STANDARD



3.2 — DOCUMENTATION STANDARD

HOW EVERY README.MD MUST LOOK LIKE:

```
/* ***** */
/*      PROJECT NAME: E_PROTOCOL;                      */
/*      BRANCHE:                                         */
/*      LOGIN: THE_NIGHTHAWK                           */
/*      DIRECTORY_NAME(XXXXXXX)  CREATED: 24/5/2099(10:00 AM)  UPDATED 24/5/2099 (10:30 PM) */
/*      FILE_NAME(README.MD)      CREATED: 24/5/2099(10:02 AM)  UPDATED  24/5/2099 (10:30 PM) */
/* ***** */

YOU ARE FREE TO WRITE ANYTHING OR USE .MD ONLY
```

GIT & COMMIT CONVENTIONS

4.1 ■ — GIT

■ HOW A GIT TREE LOOKS LIKE

▲ YOU MUST FOLLOW THIS TREE !



- always main must be the working branche
- the stable branch, never the one you code directly on. after you meet “the difinition of done” (view def of done page : 17)
- always edit the working feature on an other branch then merge them

■ GIT IGNORE

▲ ANYTHING THAT'S GENERATED, LOCAL-ONLY, OR SECRET — NEVER SOURCE CODE YOU WROTE:

- Build output (/build, /dist, compiled binaries, .o files)
- Dependencies (node_modules/)
- Editor/OS junk (.DS_Store, .vscode/ unless you deliberately want to share settings like our launch.json)
- Secrets — API keys, .env files, tokens. Never, ever commit these, even by accident. This one's not a style choice, it's a real security rule.

GIT & COMMIT CONVENTIONS

4.2 — COMMIT CONVENTIONS

■ HOW A GIT TREE LOOKS LIKE

▲ IT'S MANDATORY TO USE FOLLOWING FOMAT:

```
git commit -m "commit title" -m "commit full description "
```

▲ COMMIT TITLES:

“ type(scope): short imperative subject”

▲ YOU MUST COMMIT EVERY FILE ADDED WITH A TITLE AND A FULL DOCUMENTATION USING FOLLOWING RULES:

▲ TO SIMPLIFY YOUR COMMITS TITLE USE THIS SIMPLE STRUCTURE FOR EACH CASE:

type(scope): short imperative subject

■ HOW EVERY TEAM.MD MUST LOOK LIKE:

GIT COMMIT -m " init(scope): scaffold <project name>" -m "Initial structure for <what this circle does>. Nothing functional yet — sets up folders, norms compliance, README, NOTES."

■ HOW EVERY TEAM.MD MUST LOOK LIKE:

git commit -m " feat(net): add greeting handshake" -m "Devices now broadcast True Name and coordinates on join."

■ HOW EVERY TEAM.MD MUST LOOK LIKE:

GIT COMMIT -M "FIX(NET): MISSING SEMICOLON IN HANDSHAKE.C"

■ HOW EVERY TEAM.MD MUST LOOK LIKE:

git commit -m " docs(norms): add commit template section"

■ HOW EVERY TEAM.MD MUST LOOK LIKE:

GIT COMMIT -M " FIX(CPU): CORRECT OPCODE DECODE OFF-BY-ONE" -M "DECODER READ ONE BYTE SHORT ON MULTI-BYTE INSTRUCTIONS, CORRUPTING THE NEXT OPCODE."

STATUS/RESPONSE CODE SYSTEM

5.1 ■ — STATUS/RESPONSE CODE SYSTEM

1XX — <INFORMATIONAL>

101 PENDING — GENERIC IN-PROGRESS STATE
(SPECIFIC: 102 QUEUED, 103 STREAMING/PARTIAL)

2XX — <SUCCESS>

202 OK — GENERIC SUCCESS
(SPECIFIC: 201 CREATED, 204 NO CONTENT)

3XX — REDIRECT/REFERENCE (UNUSED SO FAR, BUT RESERVED — GOOD TO HAVE THE FAMILY EXIST EVEN EMPTY FOR NOW)

303 — GENERIC REDIRECT/POINTER-ELSEWHERE

4XX — <CALLER'S FAULT>

404 — GENERIC CLIENT ERROR
(SPECIFIC: 400 BAD REQUEST, 401 UNAUTHORIZED, 403 FORBIDDEN, 409 CONFLICT, 444 TOO MANY REQUESTS)

5XX — <OUR FAULT>

505 — <GENERIC SERVER ERROR>
(SPECIFIC: 501 NOT IMPLEMENTED, 503 UNAVAILABLE, 504 UPSTREAM TIMEOUT)

XOX — FAMILY NOT RESERVED BUT WILL BE DEFINED IN UPCOMING PROJECTS.
606 / 707/ 808/909

▲ MANY CALLS AND PROTOCOLS WILL BE DEFINED IN FOLLOWING PROJECTS.



VERSIONING RULES

6.1 — VERSIONING RULES

THE CORE IDEA — THREE NUMBERS, EACH MEANING SOMETHING SPECIFIC:

ERA . SHIFT . FIX

ERA — BUMPS ONLY WHEN SOMETHING BREAKS COMPATIBILITY — THE CONTRACT CHANGES, OLD CODE CALLING THIS WON'T WORK ANYMORE WITHOUT CHANGES ON THEIR END. THIS IS RARE AND DELIBERATE, NEVER ACCIDENTAL.

SHIFT — BUMPS WHEN YOU ADD REAL NEW CAPABILITY, BUT EVERYTHING THAT WORKED BEFORE STILL WORKS. GROWTH, NOT DISRUPTION.

FIX — BUMPS FOR BUG FIXES, TYPO CORRECTIONS, INTERNAL CLEANUP — NOTHING ABOUT HOW IT'S USED CHANGES AT ALL.

Every Echidna circle starts at 0.1.0 the moment it exists, and stays in the 0.x.x ERA for as long as it's genuinely unstable — nothing is promised, anything can break between commits, that's expected and fine. 1.0.0 isn't a birthday, it's an earned status — a circle only crosses into 1.x.x once it passes its own Definition of Done (the checklist from the norms doc). Before that: 0.x.x forever, no matter how long it takes. This stops the common trap of calling something "v1" just because time passed, when it's still genuinely half-built.

VERSIONING RULES

6.1 ■ — VERSIONING RULES

■ THE CORE IDEA — THREE NUMBERS, EACH MEANING SOMETHING SPECIFIC:

▲ CONCRETE TRIGGERS, SO YOU NEVER HAVE TO GUESS:

- Fixed a bug, no new behavior → bump FIX (0.4.2 → 0.4.3)
- Added a new command/feature, old usage still works → bump SHIFT, reset FIX to 0 (0.4.3 → 0.5.0)
- Changed something that breaks how it's called/used → bump ERA, reset SHIFT and FIX (0.5.0 → if pre-1.0, this is normal and frequent; post-1.0, this is rare and means real fair warning to anyone depending on it)

■ TAGGING IN GIT — TIES VERSION TO AN ACTUAL COMMIT, NOT JUST A NUMBER IN A FILE:

```
GIT TAG -A V0.5.0 -M "[0042] SHIFT: GREETING HANDSHAKE STABLE ACROSS ORBITS"
```

DEFINITION OF DONE

6.1 — DEFINITION OF DONE — UNIVERSAL

 EVERY CIRCLE INHERITS THIS BASELINE:

 COMPILES/RUNS CLEAN

 FOLLOWS ECHIDNA-NORMS

 README.MD EXISTS AND IS ACCURATE

 TEAM.MD HAS ZERO UNCHECKED BOXES

 DEMONSTRABLE

 NO SECRETS OR JUNK COMMITTED

 VERSION BUMPED AND TAGGED



SECURITY/TRUST BASELINE

6.1 — SECURITY/TRUST BASELINE

7 PILLERS OF ECHIDNA

■ No secrets committed, ever — API keys, private keys, tokens, passwords. Not even temporarily, not even in a private repo, not even "I'll remove it before pushing to public." .gitignore handles this structurally (section on git conventions), but the rule is absolute regardless of tooling.

■ No plaintext credentials in code or config — anything sensitive lives in environment variables or a local secrets file that's gitignored, never hardcoded.

■ Nothing is trusted by default — this one's Echidna-specific and ties straight to your Access ID work: no device, no claimed identity, no incoming data is trusted just because it says who it is. Trust is earned through the crypto proof, every time, no exceptions carved in "just for now."

■ Fail closed, not open — when something can't be verified (a signature doesn't check out, a handshake fails, a permission is ambiguous), the default behavior is to refuse, not to proceed cautiously. A system that fails open is a system that's one bug away from a real breach.

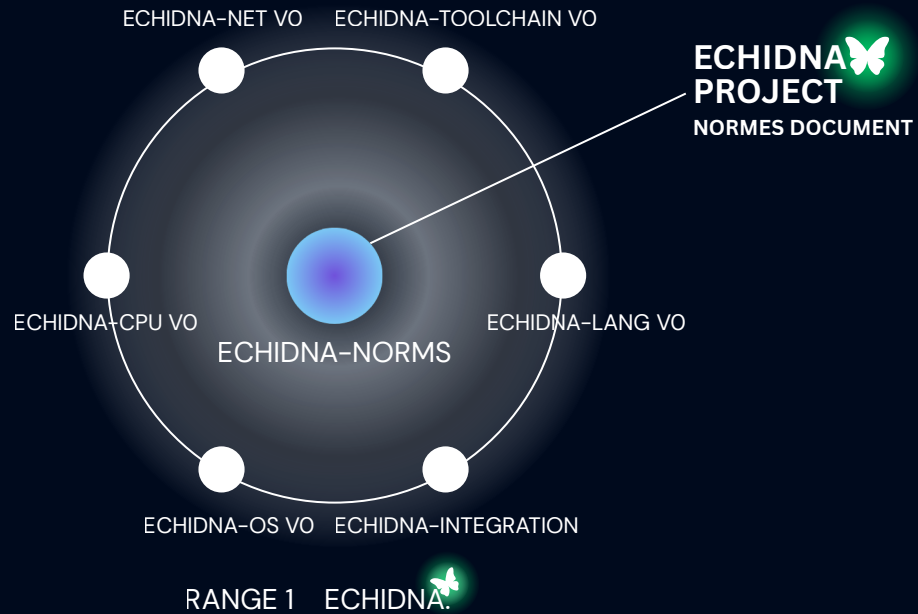
■ Every external input is hostile until proven otherwise — user input, network payloads, file contents. No assuming good faith at the parsing layer, even from your own future code.

■ No unverified dependency gets pulled in silently — anything Echidna borrows from outside (per your own "85% my own norms, borrowed help is fine" philosophy) gets checked, not blindly trusted because it's popular.

■ Logs never contain secrets — debugging output, error messages, crash dumps. A leaked log shouldn't leak a key.

ECHIDNA PROJECT

THE HOLIEST GRAPH EVER



V : 1
MAXIMUM: RANGE 1

YOU ARE AT “ECHIDNA-NORMS”

▲ NOTICE !

NOTICE THAT THIS DOCUMENT SERVES AS A GENERAL NORM AND PHILOSOPHY BASELINE. IT CAN BE ADAPTED FOR SPECIFIC PROJECTS AND MAY BE EDITED ACCORDINGLY. YOUR REVIEWS AND CONTRIBUTIONS WILL HELP US MAKE EVERYTHING MORE FUNCTIONAL AND EFFECTIVE.



ECHIDNA-NET NEXT?

OBSCURA
PROTOCOL

ECHIDNA
NET

